BRAINHACK 2026

UPDATES FOR ROBOVERSE DRONE CHALLENGE

Final Competition

Scenario

- Your team receives intelligence of the enemy's transport convoy travelling routes. Your mission is to gather information on the convoy.

Objective

- To gather intelligence on all targets of the convoy.

Challenge

The challenge will be carried out in 2 stages:

1. Reconnaissance (University teams only)
2. Deployment & Ambush

Competition Prize Awards

Pre-University Category

- 1st Place: $1800
- 2nd Place: $1300
- 3rd Place: $900

University Category

- 1st Place: $1800
- 2nd Place: $1300
- 3rd Place: $900

CHALLENGE STAGES

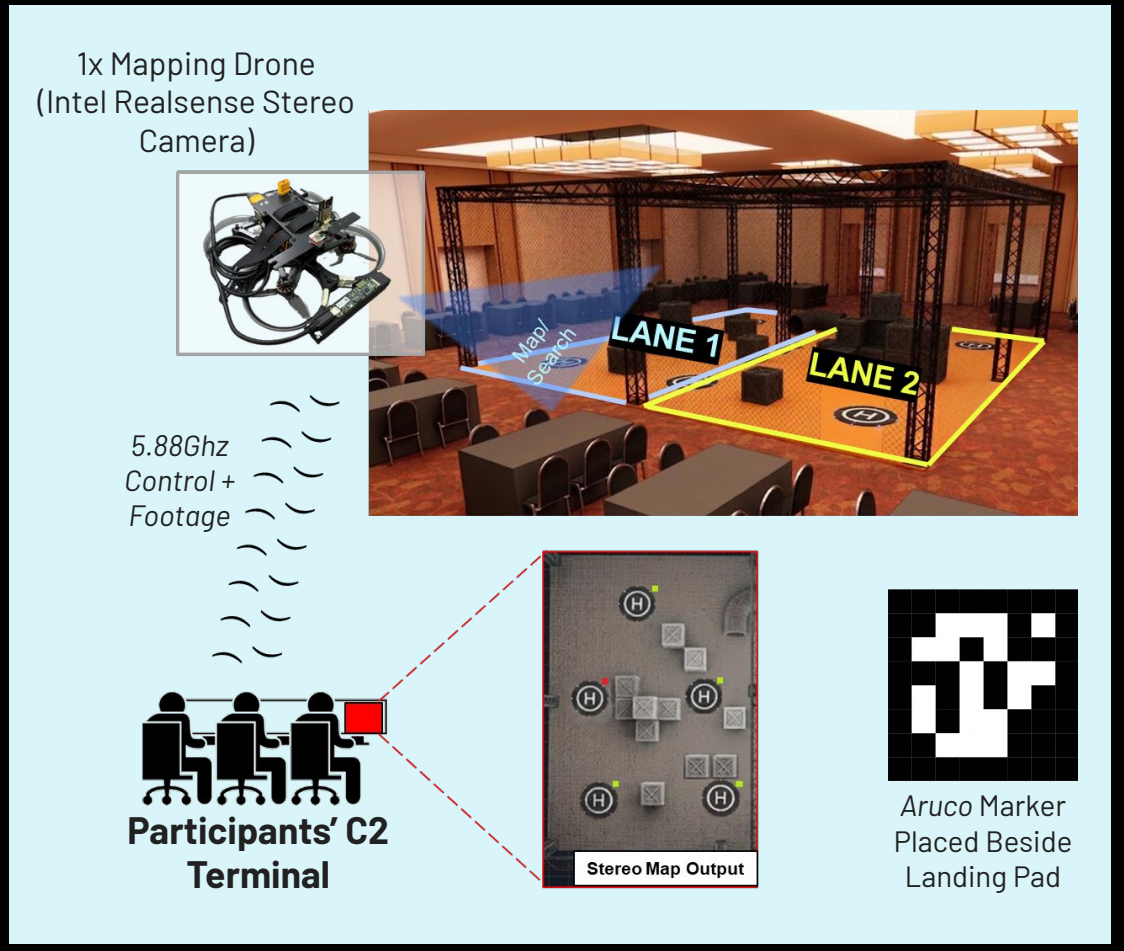
Challenge One (University teams Only)

Gameplay

- A Mapping Drone with Stereo Camera will be deployed to map the arena.
- The arena comprises obstacles and a number of Drone Landing Pads, each attached with an Aruco Marker.
- The Mapping Drone will return a top-down depth map, and images of the Aruco Marker. Teams are expected to decipher the Aruco Marker to check if each landing site is valid for landing.

Evaluation Criteria

- Teams will be awarded points based on their understanding of the concept, mapping speed, and the accuracy of identifying landing sites as valid or invalid.



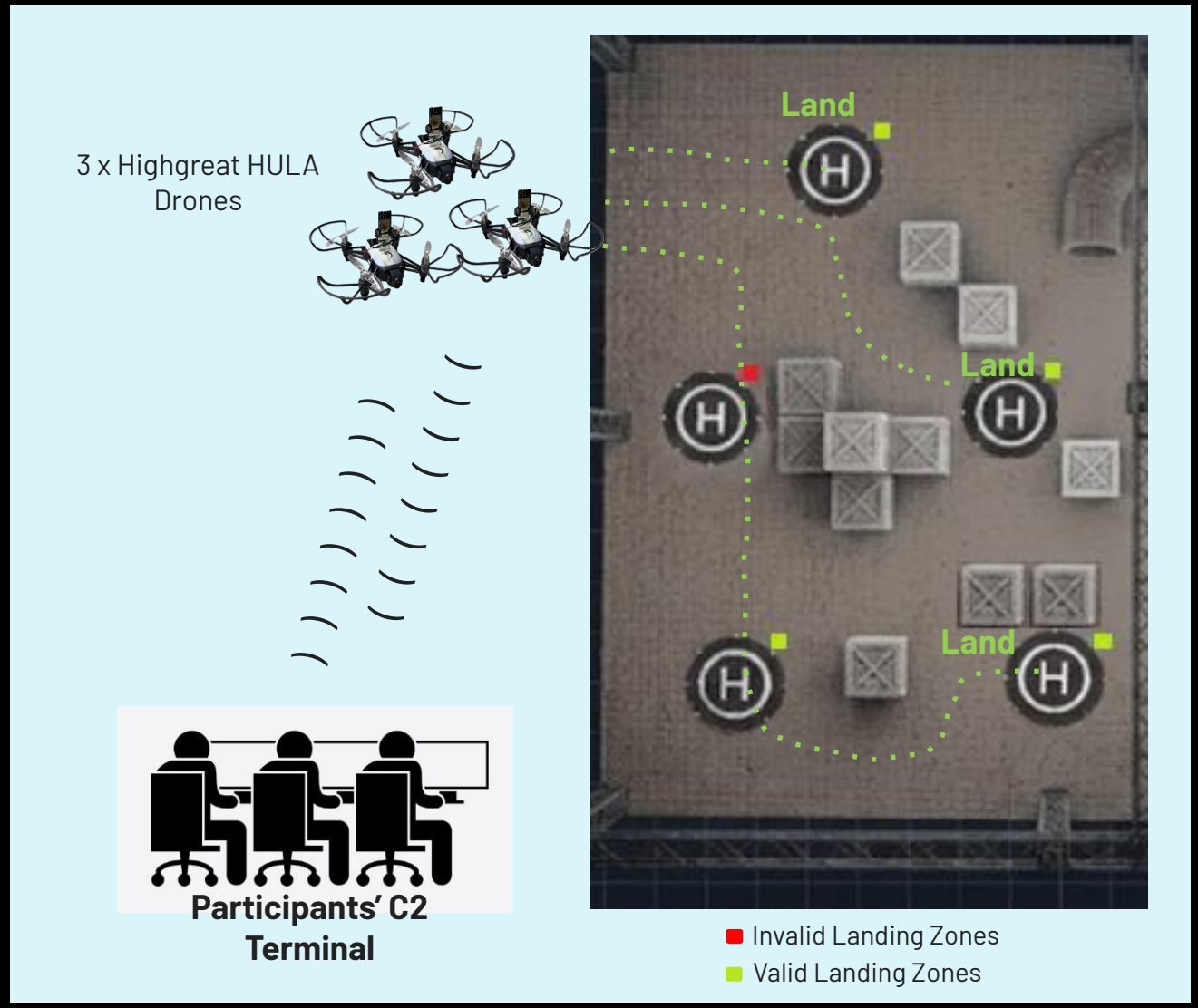
Challenge Two

Gameplay

- Based on the mapping information (will be provided), the team can strategize and select 3 of the available landing zones.
- The team will then launch 3 HULA drones from the C2 to land on the designated landing areas to prepare for ambush.

Evaluation Criteria

- Points will be awarded for successful and accurate landings on the designated zones, in the least amount of time.



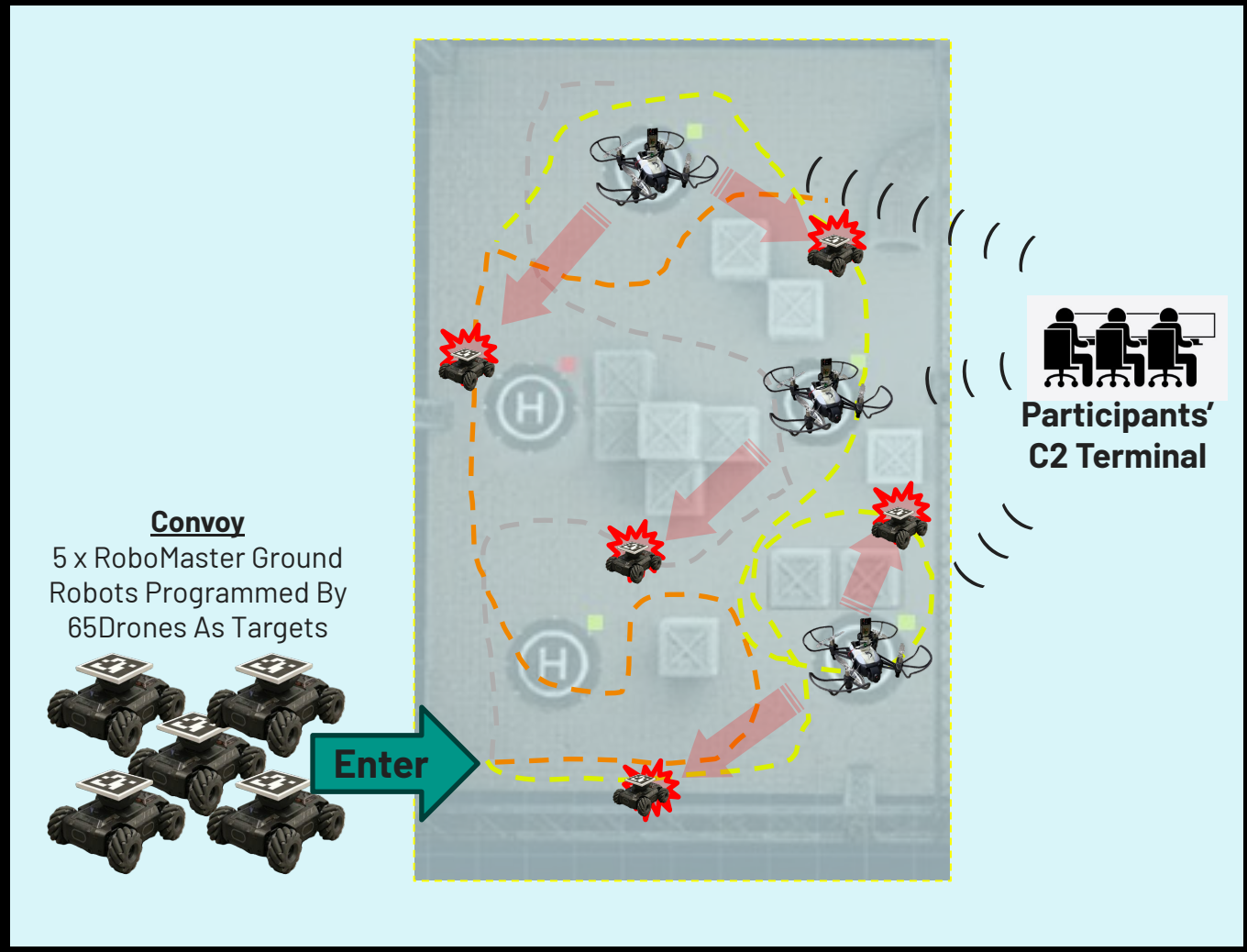
Challenge Two (cont'd)

Gameplay

- The convoy will be launched into the drone cage and these robots will loiter for a period of time.
- During this time, the team will launch the 3 HULA drones to search for these ground robots.

Evaluation Criteria

- Points will be awarded for successful and accurate snapshots of the ground robots. Teams that complete the task in less time will earn more points.



TECHNICAL INFORMATION

Mapping Drone

Mapping Drone Control and Data Access

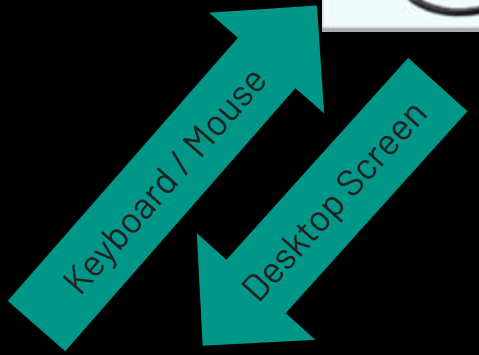
- The mapping drone's onboard computer runs the codes to control its movement and capture video from the depth camera (Realsense).
    - The mapping drone's onboard computer runs Ubuntu 22.04, with ROS2 and OpenCV installed.

Remote Access

- Teams will remotely access the mapping drone's computer through the C2 Terminal using NoMachine to execute codes and control the drone.

Programming Options

- Teams can utilize Python or C++ code on the mapping drone to access:
    - Depth Camera images: using pyrealsense2 or ROS2
    - Drone's data: using MAVSDK Python or ROS2
    - UWB data: using the provided Python class or ROS2
        - UWB data provides north-east position of the drone in the Arena.



Coding the Mapping Drone Navigation

Controlling the Mapping Drone

- To control the mapping drone, your code must use the offboard mode, as per the Qualifiers.
- You can use MAVSDK Python, which you should be familiar with.
- Alternatively, you can use ROS2, and the necessary files are available on the C2 terminal.

Recommended Approach

- We strongly recommend controlling the drone's movement using NED position commands. However, using velocity commands with UWB is more efficient.

Utilizing UWB Data

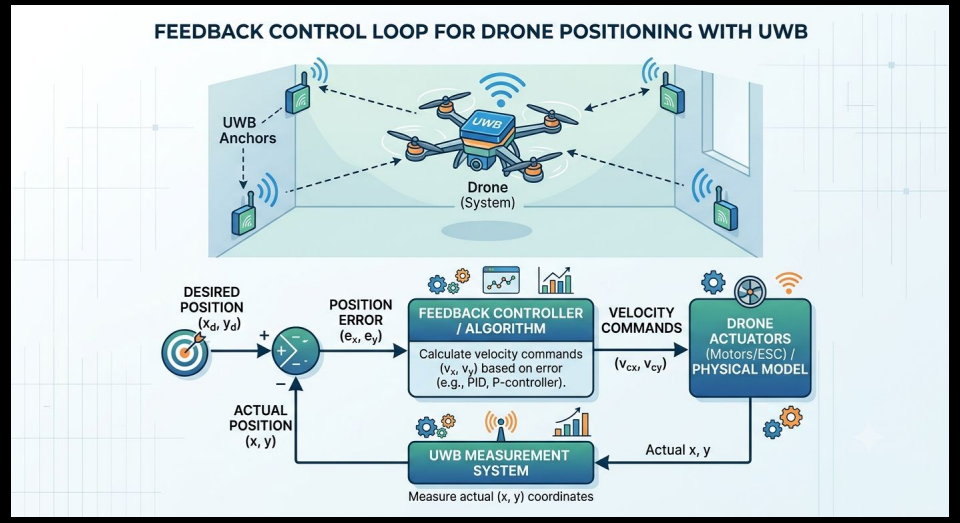
- The Mapping Drone is equipped with a UWB tag that provides its location. Your code can access the UWB topic to obtain the drone's x-y coordinates without using ROS2.

Implementation

Your code should:

- Poll the UWB position
- Calculate the error between the desired position and the current position
- Send velocity commands to correct the error
- Reference Code

For guidance, refer to kolomee.py available on the Discord channel.



Using Realsense in your python code

Mapping and Detection with Pyrealsense2

- As your drone flies to a location, perform mapping and detection using the pyrealsense2 Python library to capture RGB, depth, and point cloud data in your Python code.

Image Acquisition

- Unlike Gazebo, which streams images to your code, pyrealsense2 requires your code to actively call and retrieve images (see lines 17 and 18).
- For guidance, refer to the following example codes available on the Discord channel:
  - *generateTopDown.py*: sample code for generating a top-down occupancy grid
  - *getDepth.py*: example code for capturing depth images and calculating actual pixel distances
  - *getRGB.py*: example code for capturing RGB images
  - *getInfra.py*: useful for cameras without RGB but with infrared left and right lenses
  - *getSyncDepthColor.py*: example code for synchronized depth and color images

```python
import pyrealsense2 as rs
import numpy as np

pipeline = rs.pipeline()
config = rs.config()

config.enable_stream(rs.stream.depth, 640, 480, rs.format.z16, 30)
config.enable_stream(rs.stream.color, 640, 480, rs.format.bgr8, 30)

pipeline.start(config)

pc = rs.pointcloud()

try:
    while True:
        frames = pipeline.wait_for_frames()

        depth_frame = frames.get_depth_frame()
        color_frame = frames.get_color_frame()

        if not depth_frame or not color_frame:
            continue

        points = pc.calculate(depth_frame)

        vertices = np.asarray(points.get_vertices())

        print(f"Point count: {len(vertices)}")

finally:
    pipeline.stop()
```

Object Detection using NPU on Mapping Drone

YOLO Object Detection with NPU Acceleration

- The mapping drone's compute module supports YOLO object detection, which can be accelerated by the Neural Processing Unit (NPU) to achieve speeds of approximately 50fps.

Exporting Custom Models to RKNN Format

- To utilize the NPU for YOLO object detection, you need to export your custom model to RKNN format. Follow these steps:
  1. Export your model to ONNX format using *convertyolotoonnx.py*.
  2. Convert the ONNX file to RKNN format using *converttorknn.py*.
- These files are available on the Discord channel.
- Refer to the following example codes to learn how to perform object detection using the RKNN model:
  - *getDepthAndDetect.py*
  - *rknndecoder.py*

```python
import pyrealsense2 as rs
import cv2
import numpy as np
#from ultralytics import YOLO
from rknnlite.api import RKNNLite
from rknndecoder import decode_yolov11_rknn, draw_detections
# ------------------------------------------------
# Load YOLO model
# ------------------------------------------------


rknn = RKNNLite()

rknn.load_rknn("yolo11n.rknn")
rknn.init_runtime()
```

Swarm Drone

Controlling the Swarm of Hula Drones

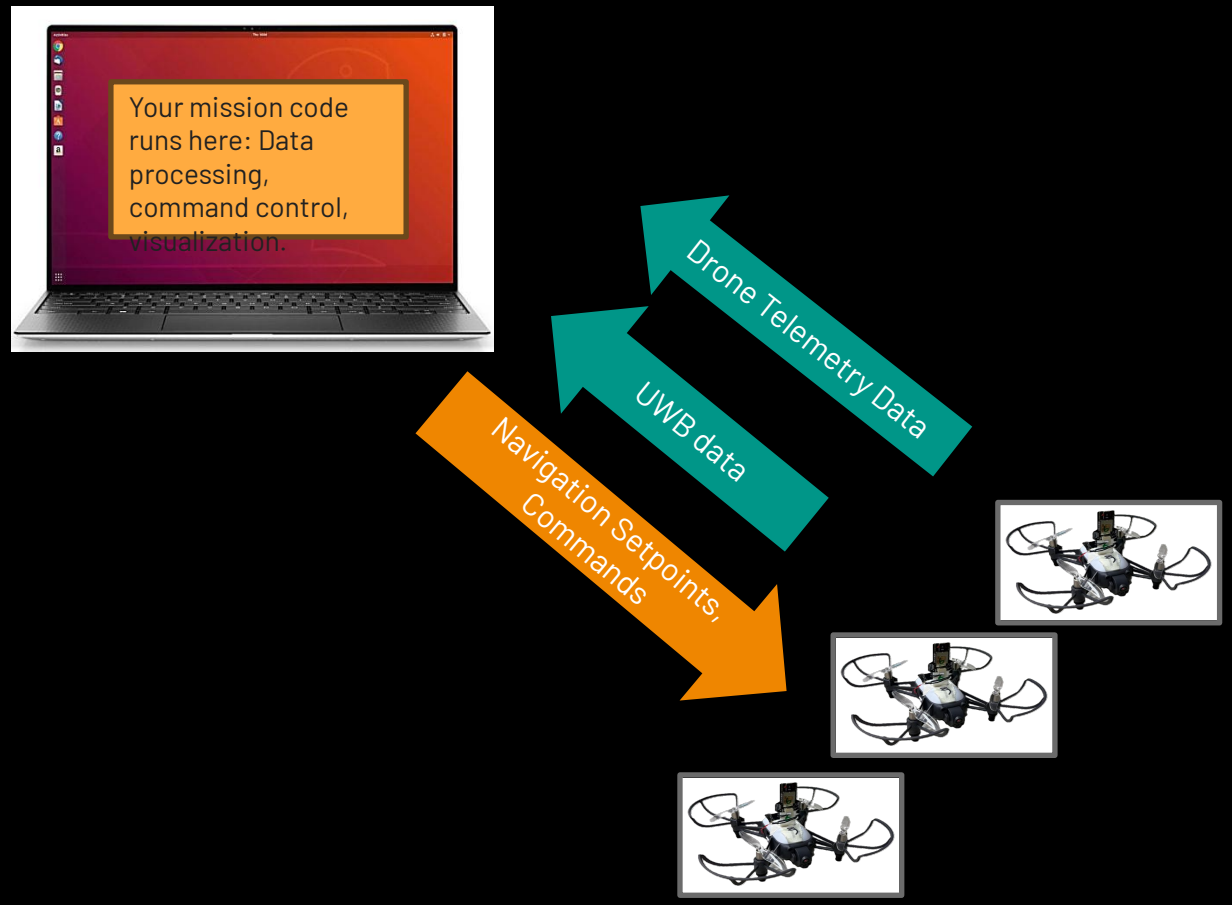
- The codes to control the swarm of Hula drones run on the C2 Terminal, a Windows laptop with an Ubuntu 22.04 Virtual Machine.

Programming Environment

- Your codes can run on the Windows environment of the C2 Terminal and utilize the following libraries:
  - *pyhulax* for controlling the Hula drones and accessing their camera images.

Accessing UWB Position

- Teams can access the UWB position of the drones using the provided UWB Python library.



THANK YOU